



BETRIEBSSYSTEME TEIL 2

IO --- Prozesse --- Multi-Processing --- Zombie-Processes --- Threads --- Deadlocks
--- Interaktion mit Python --- IPC

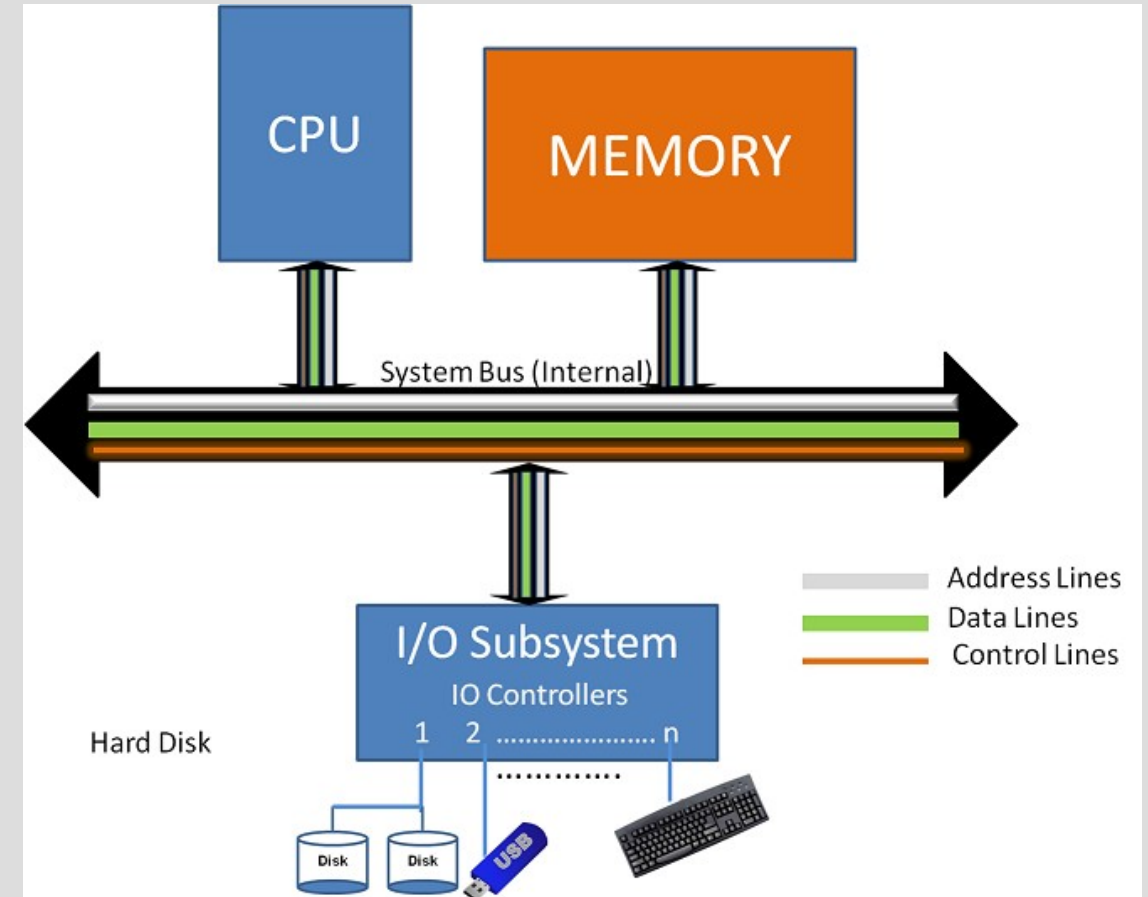
I/O

Grundfunktion jedes OS!

- Kommunikation mit Peripherie
- Kommunikation mit User

Beispiele für I/O

- Externe Speicher (SSD, USB, ...)
- Netzwerke (LAN, WLAN)
- Drucker, Scanner, ...
- Bildschirm, Tastatur, ...
- Sensoren (z.B. biometrisch)

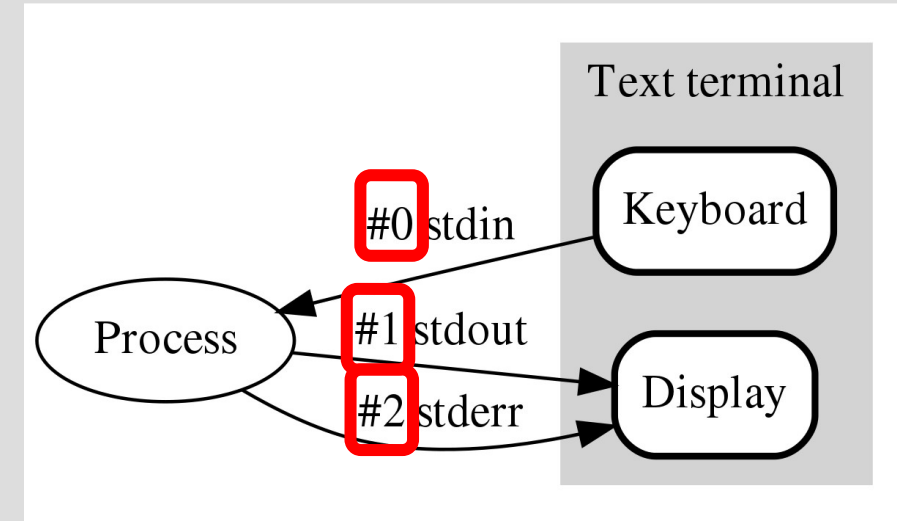


Realisierung in modernen OS

- Busse, Ports, Protokolle, Gerätetreiber, ...
oft im Kernel-Modus
- APIs zur Verwendung in Programmen

Standard-File Deskriptoren

- stdin/-out/-err werden meist **als Dateien** behandelt
- FD haben Integer-Wert:
 - Stdin: Keyboard [File Descriptor FD = 0]
 - Stdout: Bildschirm [File Descriptor FD = 1]
 - Stderr: Bildschirm [File Descriptor FD = 2]



Linux-Shell (Bash)

- Stream: stdin (< für Umleitung)

```
$ cat
This is some input from the keyboard.
^D # Drücke Strg+D, um die Eingabe zu beenden (EOF).
```

```
$ cat < input.txt
```

Linux-Shell (Bash)

- Stream: stdout (> für Umleitung in Datei, >> für Anhängen an Datei)

```
$ echo "Hello, World!" # Die Ausgabe wird auf den Bildschirm (Terminal)
Hello, World!
```

```
$ echo "Dies ist Zeile 1" > ausgabe.txt
$ echo "Dies ist Zeile 2" >> ausgabe.txt
$ cat ausgabe.txt
Dies ist Zeile 1
Dies ist Zeile 2
```

Linux-Shell (Bash)

- Stream: stderr (2> für Umleitung in Datei, 2>&1 lenkt auch stdout um)

```
$ cat non_existent_file.txt  
cat: non_existent_file.txt: No such file or directory
```

```
$ cat non_existent_file.txt 2> error.txt
```

```
$ ls nicht_existierendes_verzeichnis 2>&1 > fehler_und_ausgabe.txt  
$ cat fehler_und_ausgabe.txt  
ls: cannot access 'nicht_existierendes_verzeichnis': No such file or direct
```

I/O

Mit Python-Programmen

```
$ python streams.py < numbers.txt
```

```
$ python streams.py < numbers.txt > output.txt
```

```
$ python streams.py < input.txt > output.txt 2>&1
```


I/O

Mit Windows-PS

Stream #	Description	Introduced in	Write Cmdlet
1	Success Stream	PowerShell 2.0	<code>Write-Output</code>
2	Error Stream	PowerShell 2.0	<code>Write-Error</code>
3	Warning Stream	PowerShell 3.0	<code>Write-Warning</code>
4	Verbose Stream	PowerShell 3.0	<code>Write-Verbose</code>
5	Debug Stream	PowerShell 3.0	<code>Write-Debug</code>
6	Information Stream	PowerShell 5.0	<code>Write-Information</code>
*	All Streams	PowerShell 3.0	

```
PS> ls > my_files.txt
```

```
PS> ps >> my_files.txt
```

```
PS> cat numbers.txt | python3.exe .\streams.py
```

```
PS> Get-Content numbers.txt | python3.exe .\streams.py
```

```
PS> cat numbers.txt | python3.exe .\streams.py > output.txt
```

```
PS> cat numbers.txt | python3.exe .\streams.py 2>&1 log.txt > output.txt
```

AUFGABE

Arbeitsblatt AB_IO_FD_Streams

Tipp Exceptions:

https://www.w3schools.com/python/python_ref_exceptions.asp

PROZESSVERWALTUNG

Arbeitsblatt AB_IO_FD_Streams

WAS IST EIN PROZESS?

- Vorgang einer algorithmisch ablaufenden Informationsverarbeitung
- Prozess = Programm in Ausführung
- „exklusive“ Nutzung: Ressourcen (CPU, RAM, Ressourcen)
- Eigener Speicherbereich: Geschützt vor Zugriff anderer Prozesse
- Eigene Kommunikation zwischen Prozessen nötig
- Task = Prozess

PROZESSE IN BETRIEBSSYSTEMEN

WINDOWS

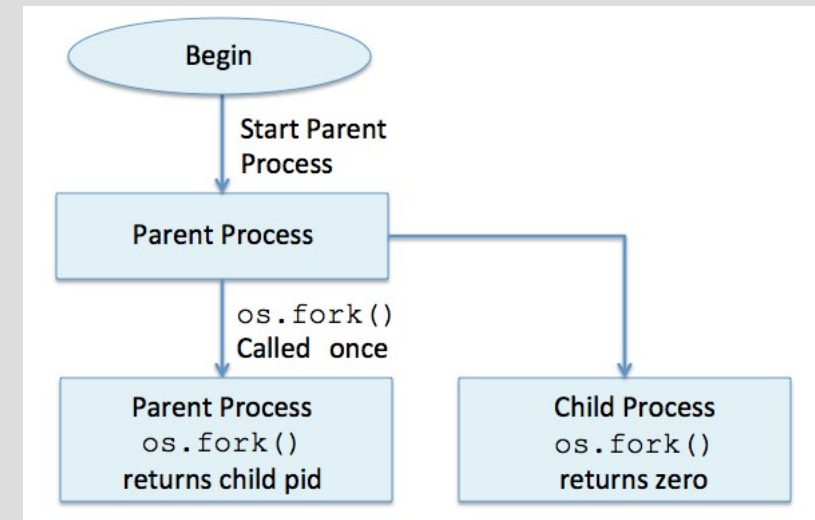
- Keine Hierarchie
- CreateProcess, ExitProcess, TerminateProcess
- Keine Datenübernahme

Prozess starten in Python:

`os.system()` .. Startet Prozess über neue Konsole
`os.exec()` .. Ersetzt aktuellen Prozess

LINUX

- Eltern-/Kind-Prozesse
- `os.fork()`
- Daten übernommen



PROZESSE IN PYTHON

- Eigene Libraries: multiprocessing bzw. subprocess
<https://docs.python.org/3/library/multiprocessing.html>
- Klassen für
 - Prozesse: Process, Pool
 - Kommunikation zwischen Prozessen: Queue, Pipe
 - Synchronisation: Lock, Barrier, Semaphore
- Drei Wege um Prozesse zu erzeugen:
 - spawn: Unix, Windows, macOS -> neuer Python-Interpreter wird gestartet
 - fork: Unix (default) -> os.fork() wird verwendet
 - forkserver: Unix -> zuerst ein Eltern-Prozess, dann werden Kinder mit os.fork() erzeugt

PROZESS STARTEN IN PYTHON

```
1 import subprocess
2 command = 'echo "Hello, World!'"
3 process = subprocess.Popen(command, shell=True)
4 # Prozess-ID abrufen
5 pid = process.pid
6 print(f"Prozess wurde gestartet. PID: {pid}")
7
8 process.wait()
```

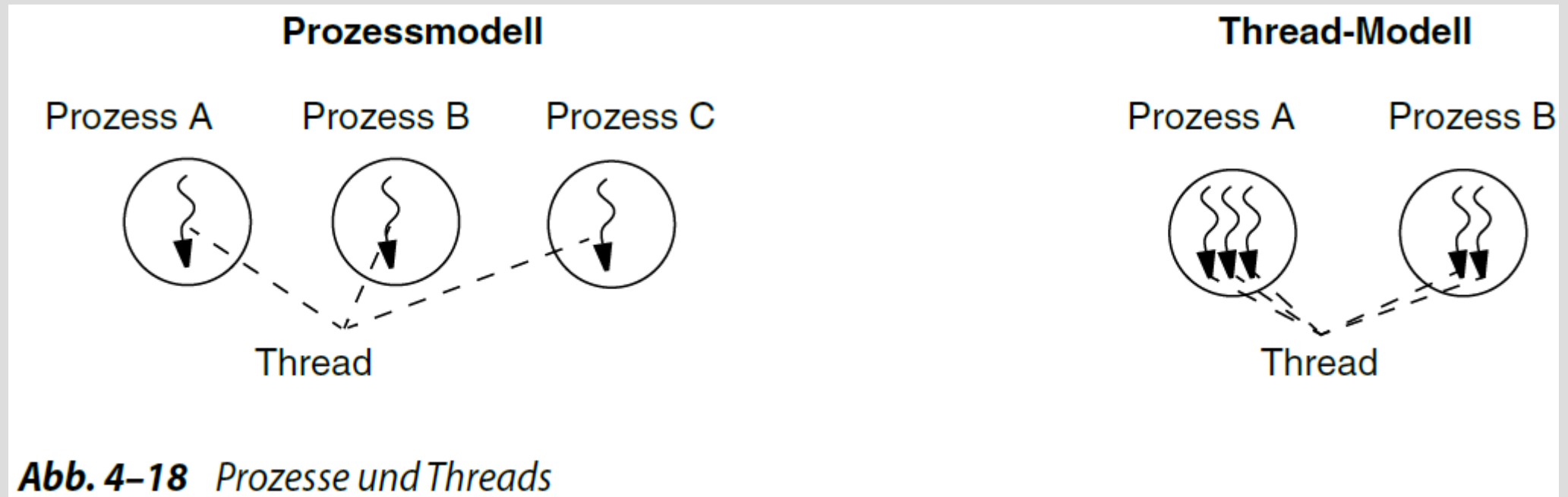
<

Kommandozeile ×

```
>>> %Run -c $EDITOR_CONTENT
```

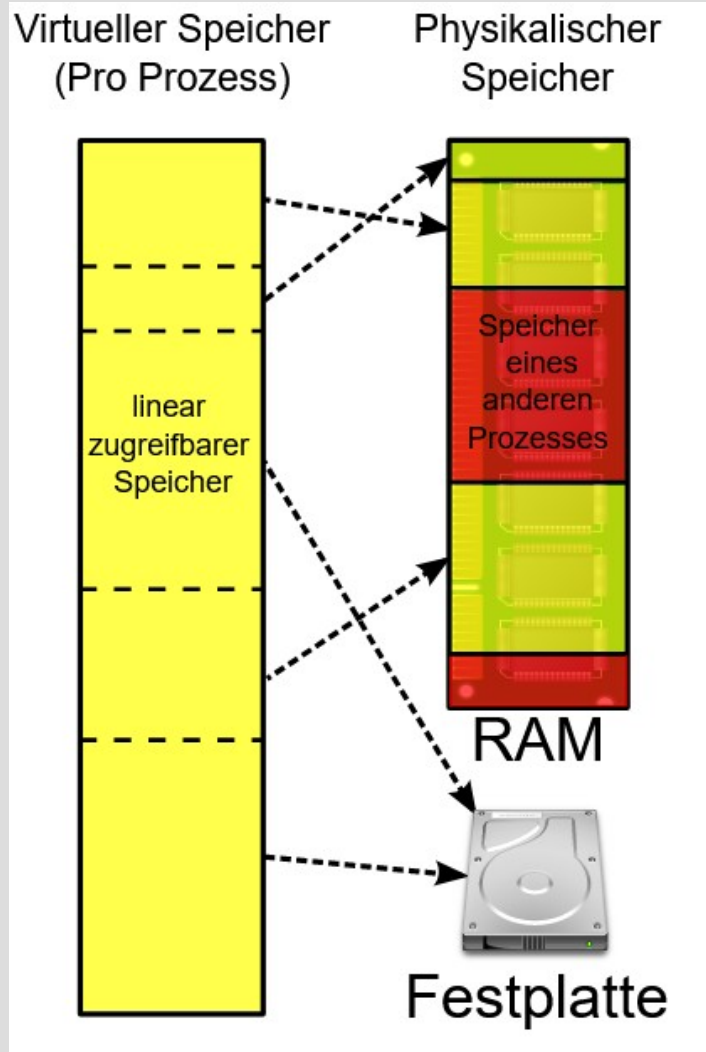
```
Prozess wurde gestartet. PID: 21976
"Hello, World!"
```

PROZESS VS. THREAD

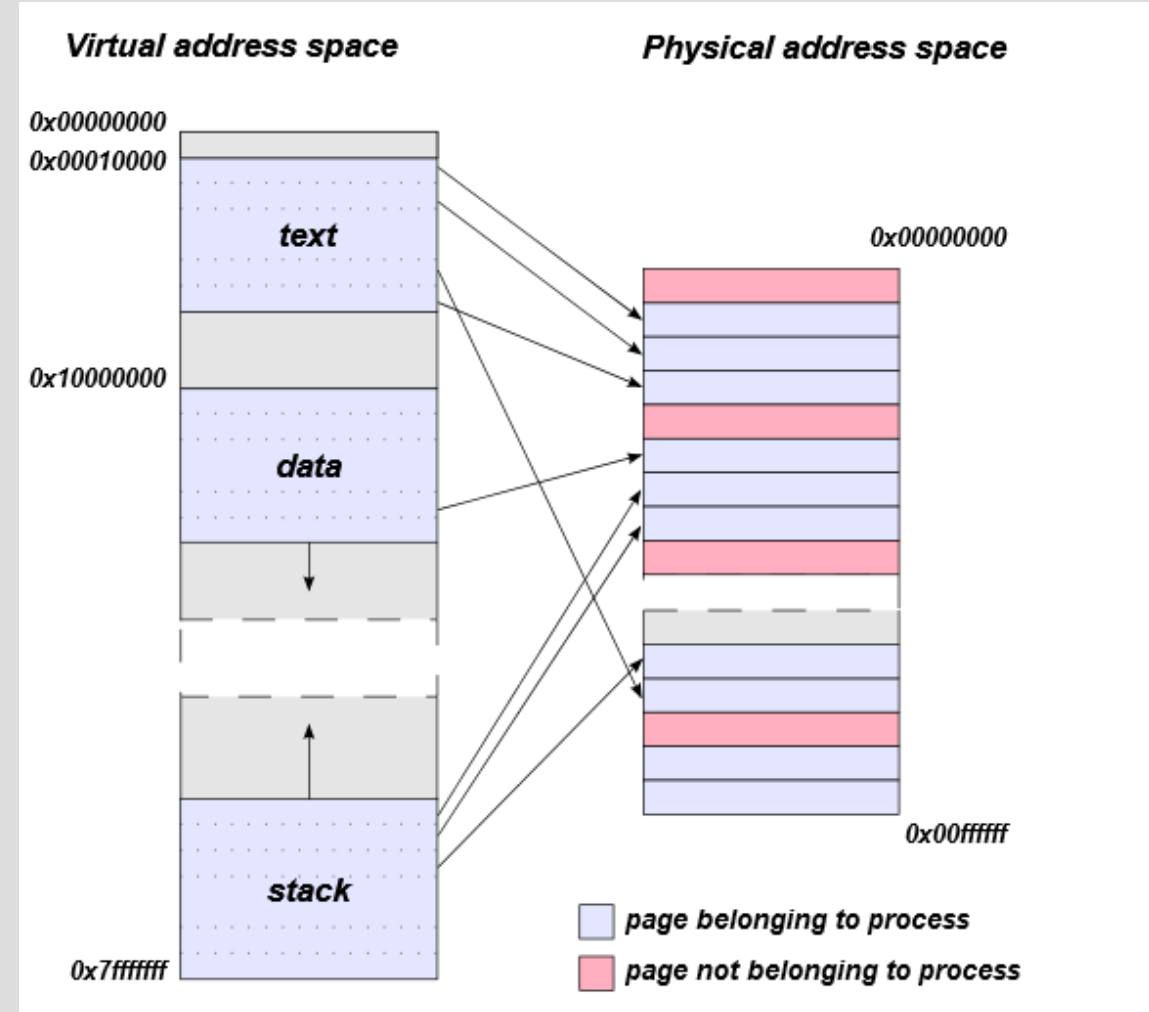


Quelle: <https://www.repetico.de/card-78404905> [10.09.2024]

PROZESSE IM ARBEITSSPEICHER



FSST 4

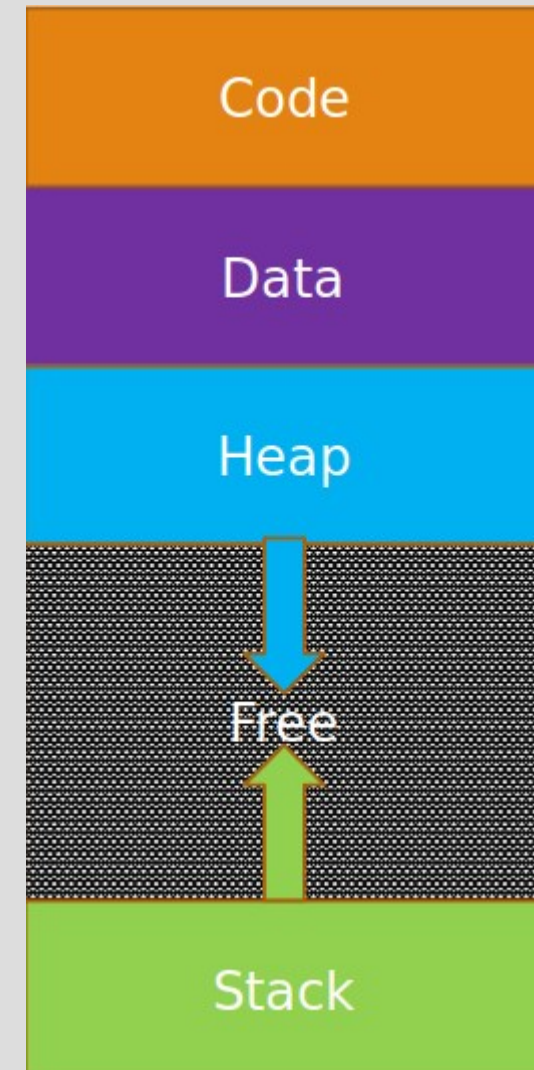


17

PROZESSE IM ARBEITSSPEICHER

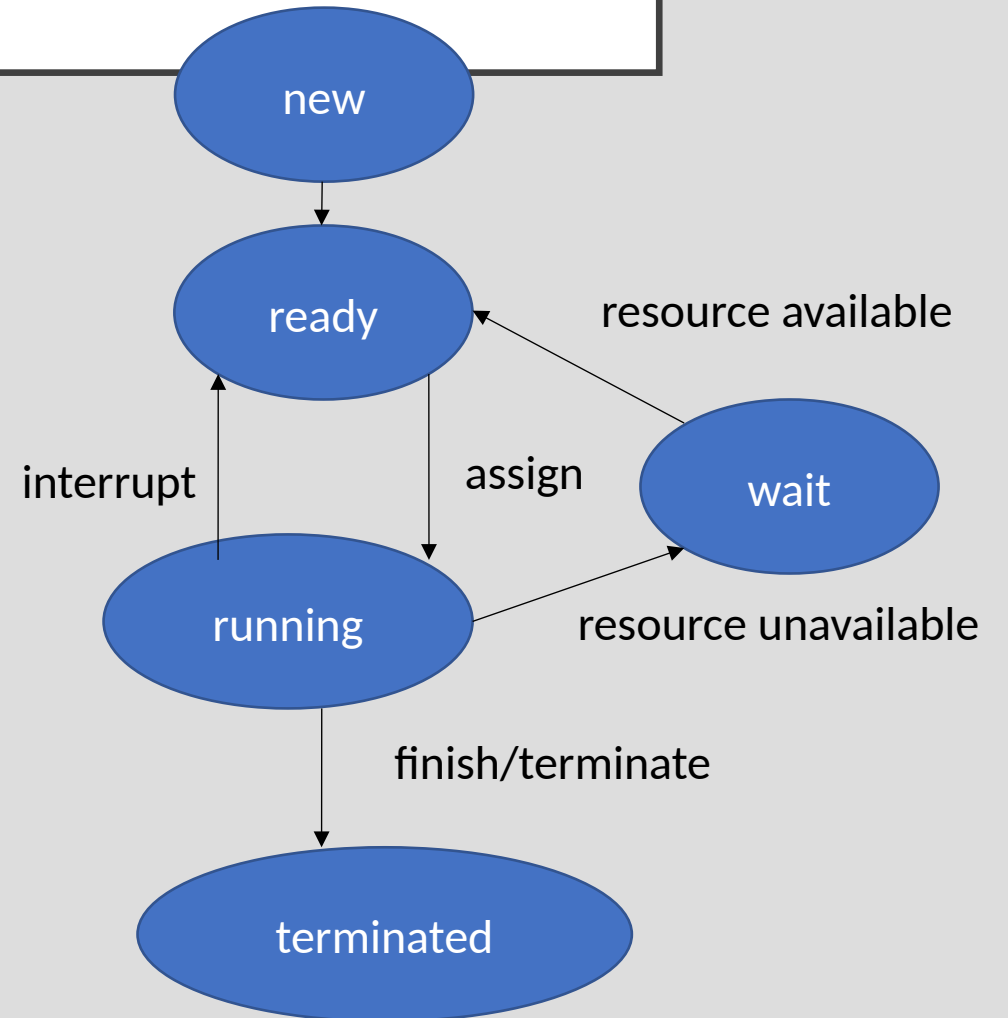
Code-Stack-Heap-Memory-Layout:

- Code: fix
 - CPU-Instruktionen des kompilierten Programmes
- Data: fix
 - Globale Daten wie z.B. globale Variablen
- Heap (variabel)
 - freier Speicherbereich des Programmes
 - z.B. zur Laufzeit generierte Variablen, Objekte
- Stack (variabel)
 - Funktionsaufrufe inkl. lokale Variablen, return-Adressen
 - => sehr groß bei rekursiven Aufrufen
- Free (variabel)
 - zwischen Stack & Heap
 - dynamische Zuweisung während der Laufzeit



PROZESS: ABLAUF

- Prozess-Erzeugung
 - Laden des Programms
 - Wichtig: Adressraum !
 - Starten des Programms
- Programmablauf
 - Immer wieder unterbrochen durch
 - Zeitmultiplex
 - Interrupts
 - E/A-Pausen
- Beendigung des Prozesses
 - Kann freiwillig oder unfreiwillig erfolgen
 - Entladen des Programms



MULTI-TASKING

EIN-PROZESSOR-SYSTEM

- **Zeit-Multiplex-Verfahren**
 - Einer nach dem anderen
 - Wer kommt zuerst?
- Prioritätsbasierte Ressourcenzuteilung

Aufteilung, welcher Prozess wann dran kommt nennt man **Scheduling**.

MEHR-PROZESSOR-SYSTEM

- **Echte Gleichzeitigkeit möglich**
- Unterscheidung:
 - Mehrere „echte“ Prozessoren / Sockel
 - Ein Prozessor mit mehreren Kernen
 - Rechnerverbund (Cluster)
- **Speicher-Verwaltung / -Zuteilung**
 - Uniform Memory Access (gemeinsames RAM für alle CPUs)
 - Non-Uniform Memory Access (eigenes RAM für jede CPU)

ANFORDERUNGEN AN SCHEDULING?

ANFORDERUNGEN AN SCHEDULING

- **Fairness**
 - gerechte Verteilung der CPU an alle Prozesse
- **Effizienz**
 - optimale CPU-/System-Auslastung
 - Reduktion von Prozesswechseln
- **Antwortzeiten**
 - minimale Antwortzeiten (interaktive System)
 - Einhalten von Antwortzeiten (Echtzeitsysteme)
- **Wait-Times**
 - möglichst gering für Hintergrundprozesse
- **Durchsatz**
 - größtmögliche Anzahl an Verarbeitungen/Zeiteinheit

PROZESS SCHEDULING STRATEGIEN

NON-PREEMTIVE (KOOOPERATIV)

- Aktive Freigabe der CPU durch den Prozess
- + geringer Verwaltungsaufwand
- - Gefahr durch „unkooperative“ Prozesse

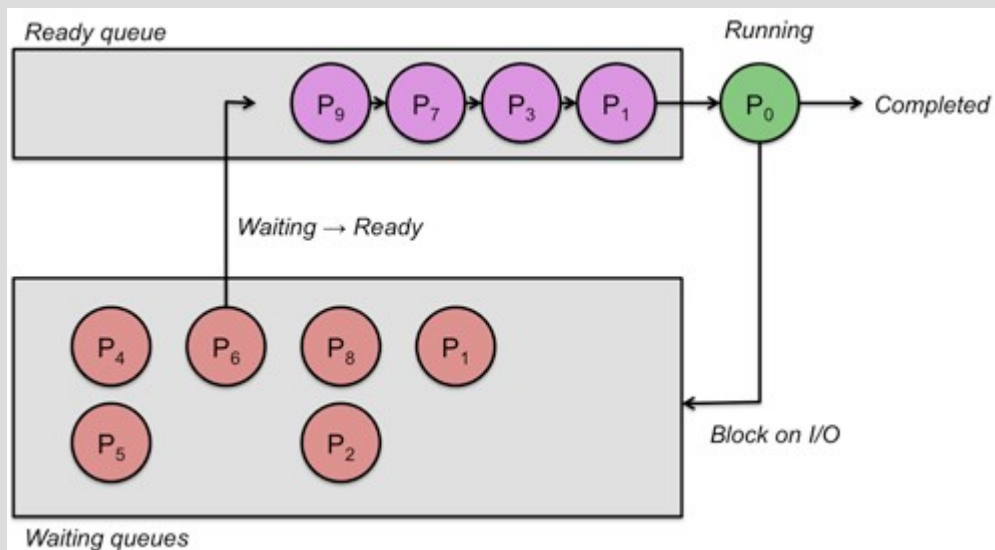
PREEMTIVE (VERDRÄNGEND)

- CPU-Freigabe durch Scheduler
- + keine Gefahr durch unkooperative Prozesse
- - erhöhter Verwaltungsaufwand



FIRST COME FIRST SERVE: FCFS, FIFO

- Neugestartete/Non-Blocking Prozesse am Ende der Warteschleife eingereiht.
- Non-Preemptive: CPU-Belegung bis Beendigung/Blocking.
- Nach Ablauf der Time Slice: Einreihung am Ende der Warteschleife

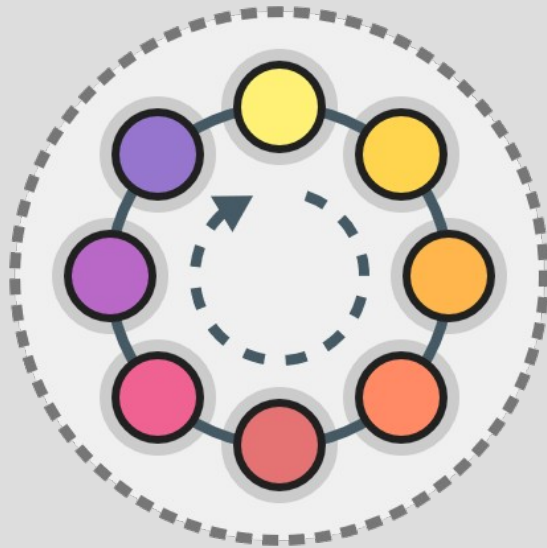


Quelle: <https://karrierebibel.de/warteschlange/>

+ wenige Prozesswechsel -> gute Systemauslastung
+ einfache Implementierung
- schlechte Antwortzeiten
(kurz laufende Prozesse durch längere blockiert)

ROUND ROBIN

- Jeder Prozess kommt eine gewisse Zeit (Timeslice) dran
- Danach stellt er sich hinten wieder an, wenn er noch nicht fertig ist
- Prozess-Stand muss gespeichert werden



+ gleichmäßige Verteilung der Ressourcen auf alle Prozesse
+ geringe Antwortzeiten bei kurzen Time Slices
- höherer Verwaltungsaufwand durch häufige Prozesswechsel

WEITERE SCHEDULING VERFAHREN

SHORTEST JOB FIRST

- Es beginnt jener Prozess mit der kürzesten Rechenzeit
- Rechenzeit ist bekannt oder wird geschätzt
- Kürzeste Warte-Zeit über alle Prozesse
- Starvation möglich

LOTTERY SCHEDULING

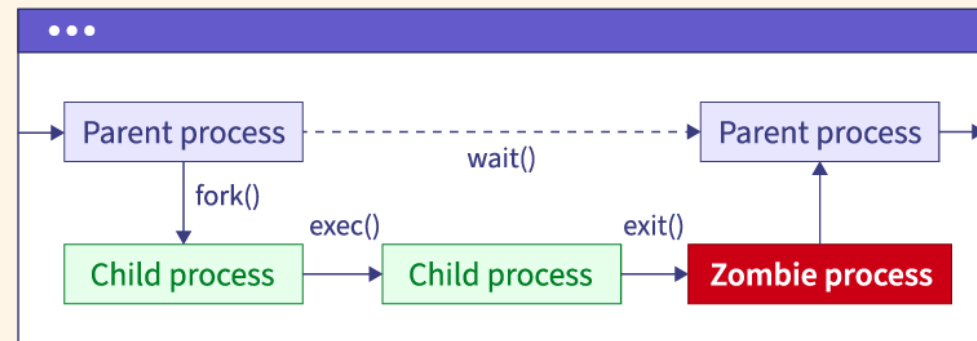
- CPU-Zuteilung erfolgt zufällig
- Jeder Prozess bekommt Lose
- Je mehr Lose, desto wahrscheinlicher, dass Prozess dran kommt

ACHTUNG: ZOMBIES

- Nur in Unix-Systemen
- Prozesse werden in der Prozess-Tabelle eingetragen
- Beendet sich ein Kind (fork), muss Eltern-Prozess weiterhin Infos abfragen können
- Danach wird Eintrag in der Prozess-Tabelle gelöscht
- Problem: Elternprozess fragt Infos nicht ab!
- Weiter Infos:



Zombie Process in Linux



ZOMBIES: BEISPIEL

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t child_pid;

    child_pid = fork(); // Create a child process

    if (child_pid < 0) {
        fprintf(stderr, "Fork failed\n");
        return 1;
    } else if (child_pid == 0) {
        // Child process
        printf("Child process executing\n");
        exit(0);
    } else {
        // Parent process
        sleep(2); // Parent process waits for 2 seconds

        // The parent process does not handle the termination of the child process
        // It does not invoke wait() or waitpid() to collect the exit status

        printf("Parent process executing\n");
        printf("Zombie process created\n");
    }

    return 0;
}
```

- Parent Prozess erstellt Child Prozess mit fork()
- Child Prozess läuft ab und wird mit exit(0) beendet
- Parent Prozess wartet 2 Sekunden und fängt exit-status vom Child Prozess nicht ab
- Prozess beendet, Parent Prozess hat exit-status nicht abgefangen → Zombie

ZOMBIES: LÖSUNG

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t child_pid;

    child_pid = fork(); // Create a child process

    if (child_pid < 0) {
        fprintf(stderr, "Fork failed\n");
        return 1;
    } else if (child_pid == 0) {
        // Child process
        printf("Child process executing\n");
        exit(0);
    } else {
        // Parent process
        sleep(2); // Parent process waits for 2 seconds

        // Parent process waits for the termination of the child process
        wait(NULL);

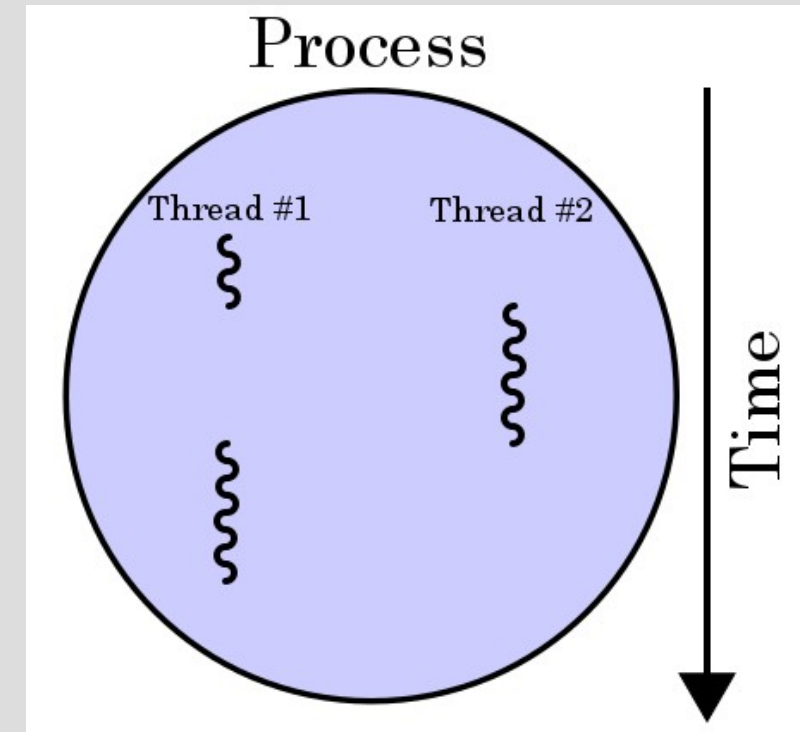
        printf("Parent process executing\n");
        printf("Zombie process handled\n");
    }

    return 0;
}
```

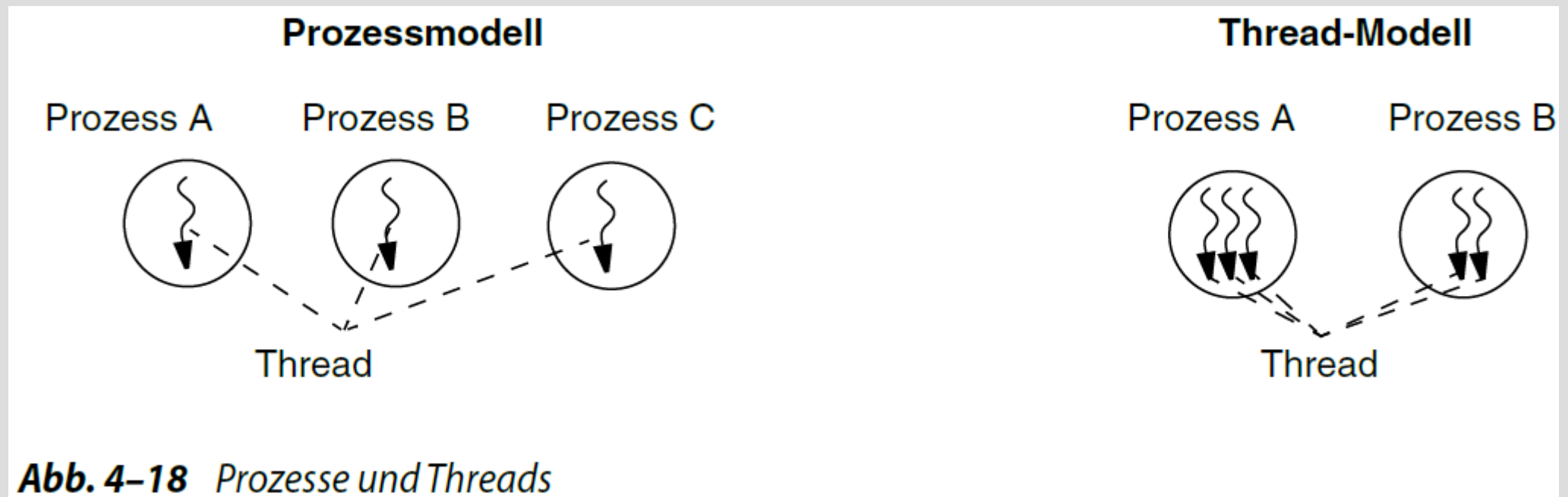
- wait() verwendet, um auf die Beendigung des Child Prozesses zu warten
- → Kein Zombie
- OS räumt regelmäßig Zombie-Prozesse auf, trotzdem: bei vielen Zombies evt. Performance-Probleme

THREADS – EIN FADEN

- Sequenz von Anweisungen (vgl. Funktion)
- Manchmal sinnvoll, mehrere Threads in einem Prozess zu haben
- Verschiedene Befehlsfolgen, laufen oft parallel in verschiedenen Kernen / Prozessoren
- gleicher Adressraum, gleiche Ressourcen möglich
- Kommunikation zwischen den Threads desselben Prozesses ist leicht möglich
- Moderne Prozessoren unterstützen Multi-Threading



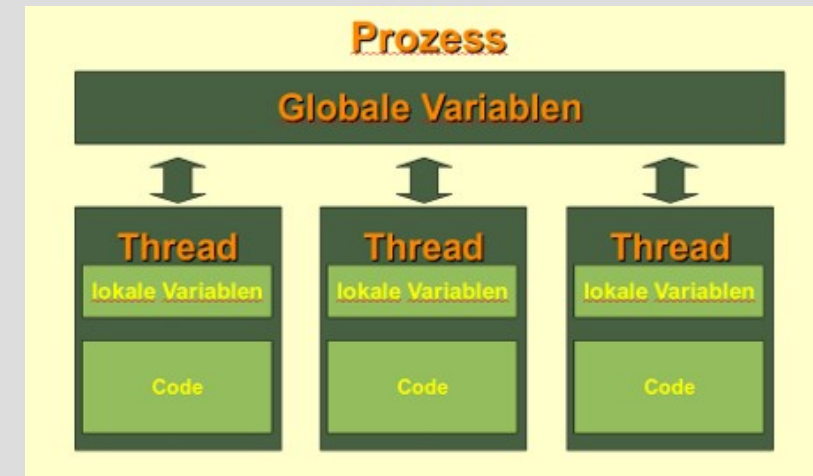
PROZESS VS. THREAD



Quelle: <https://www.repetico.de/card-78404905>

MULTI-THREADING

- gleichzeitiges Abarbeiten mehrerer Threads eines Prozesses
- Ein-Prozessor-Systeme:
 - Softwareseitiges Multi-Threading
 - Sequenzialisierung, Zeit-Multiplexing
- Systeme mit mehreren Prozessoren/Kernen:
 - Echte Gleichzeitigkeit ist möglich
 - Muss von der Software unterstützt werden



Quelle: <https://www.python-kurs.eu/threads.php>

THREADS IN PYTHON 3

- Library threading
 - Laufen nicht wirklich parallel:
Global Interpreter Lock (GIL)
 - Threads sind **Objekte**
 - Erzeugen: `threading.Thread(...)`
 - Starten: `t.start()`
 - Warten: `t.join()`
- Library `concurrent.futures`
 - einfachere Handhabung
 - Für threading und processing verfügbar

```
1  import threading
2  import time
3
4  def do(name):
5      print("hallo " + name)
6      time.sleep(5)
7      print("good bye " + name)
8
9
10 t1 = threading.Thread(target=do, args=("alexander",))
11 t2 = threading.Thread(target=do, args=("richard",))
12 t1.start()
13 t2.start()
```

CONCURRENT FUTURES

```
from concurrent.futures import ThreadPoolExecutor
import time

def worker(number):
    print(f"Calculating the result for number {number}")
    time.sleep(2)
    return number ** 2

# Create 2 threads in one pool
pool=ThreadPoolExecutor(2)
work1 = pool.submit(worker,7)
work2 = pool.submit(worker,9)
work3 = pool.submit(worker,5)
work4 = pool.submit(worker,8)
work5 = pool.submit(worker,1)
```

CONCURRENT FUTURES

```
# .done() checks if the work is finished
print(work3.done())

# use .done() to check if result is ready
if work3.done():
    print(work3.result())
else:
    print("no results yet")
time.sleep(5)
if work3.done():
    print(work3.result())
else:
    print("no results yet")

# .result() waits for the work to finish
print(work3.result())
```

```
from concurrent.futures import ThreadPoolExecutor
import time

def worker(number):
    print(f"Calculating the result for number {number}")
    time.sleep(2)
    return number ** 2

# 2 worker threads are executed at each instance
pool=ThreadPoolExecutor(2)
work1 = pool.submit(worker,7)
work2 = pool.submit(worker,9)
work3 = pool.submit(worker,5)
work4 = pool.submit(worker,8)
work5 = pool.submit(worker,1)

# Do not allow more threads
pool.shutdown()

work6=pool.submit(worker,9) # Error
```

CALC EXAMPLE

```
import threading
import time

def
calc_square(num):
    print("Thread c1
started.")
    time.sleep(2)
    print(num*num)

def calc_cube(num):
    print("Thread c2
started.")

print(num*num*num)
```

```
c1 = threading.Thread(target=calc_square,
args=(10,))
c2 = threading.Thread(target=calc_cube,
args=(10,))

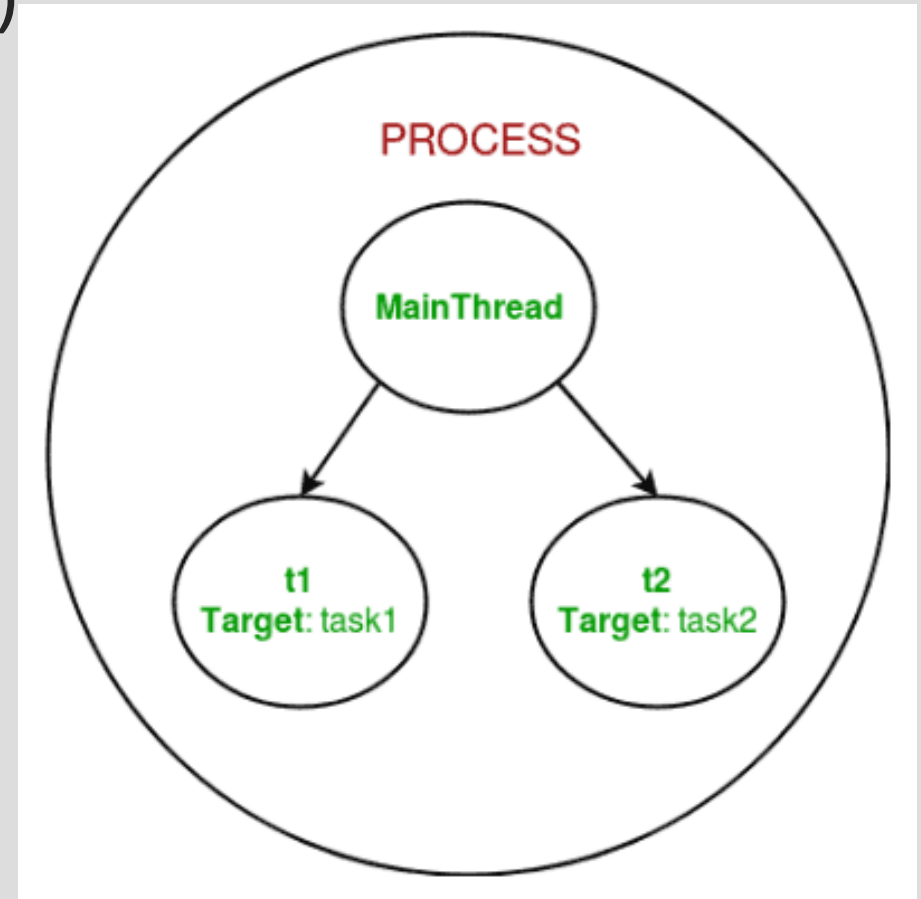
c1.start()
c2.start()

c1.join()
print("c1 done.")
c2.join()
print("c2 done.")
print("Done.")
```

PROCESSING VS. THREADING

- Ein Prozess hat zumindest einen (main) Thread
- Innerhalb eines Prozesses können beliebige Threads erstellt werden
 - Diese haben alle die **gleiche PID**
- Beispielcode nächste Folie liefert:

```
ID of process running main program: 2828
Main thread name: MainThread
Task 1 assigned to thread: t1
ID of process running task 1: 2828
Task 2 assigned to thread: t2
ID of process running task 2: 2828
```



PROCESSING VS. THREADING

```
import threading
import os
```

```
def task1():
    print("Task 1 assigned to thread:
{}").format(threading.current_thread().name
))
    print("ID of process running task 1:
{}").format(os.getpid()))

def task2():
    print("Task 2 assigned to thread:
{}").format(threading.current_thread().name
))
    print("ID of process running task 2:
{}").format(os.getpid()))
```

```
if __name__ == "__main__":
```

```
    print("ID of process running main
program: {}".format(os.getpid()))
```

```
    print("Main thread name:
{}").format(threading.current_thread().name
))
```

```
    t1 = threading.Thread(target=task1,
name='t1')
    t2 = threading.Thread(target=task2,
name='t2')
```

```
    t1.start()
    t2.start()
```

```
    t1.join()
    t2.join()
```

PROCESSING VS. THREADING

```
1 import multiprocessing
2 import os
3
4 count = 0
5
6 def run(name):
7     global count
8     pid = os.getpid()
9     count = count + 1
10    print("Process %s with id=%d (count=%d)."
11          % (name, pid, count))
12
13 if __name__ == '__main__':
14     run("parent")
15     for i in range(5):
16         p = multiprocessing.Process(
17             target=run,
18             args=("child #%d" % i,))
19         p.start()
20         p.join()
```

Process parent with id=7268 (count=1).
Process child #0 with id=4252 (count=1).
Process child #1 with id=26988 (count=1).
Process child #2 with id=28732 (count=1).
Process child #3 with id=27296 (count=1).
Process child #4 with id=27460 (count=1).

```
1 import threading
2 import os
3
4 count = 0
5
6 def run(name):
7     global count
8     pid = os.getpid()
9     count = count + 1
10    print("Thread %s with id=%d (count=%d)."
11          % (name, pid, count))
12
13 if __name__ == '__main__':
14     run("parent")
15     for i in range(5):
16         t = threading.Thread(
17             target=run,
18             args=("child #%d" % i,))
19         t.start()
20         t.join()
```

Thread parent with id=27036 (count=1).
Thread child #0 with id=27036 (count=2).
Thread child #1 with id=27036 (count=3).
Thread child #2 with id=27036 (count=4).
Thread child #3 with id=27036 (count=5).
Thread child #4 with id=27036 (count=6).

AUFGABE: RATESPIEL MIT THREADS

- „Guess the Number“ Game mit zwei Threads
 - Thread 1: Stoppuhr (10 Sekunden)
 - Thread 2: User gibt eine Zahl ein ($0 < \text{zahl} < 100$)
- Wenn Thread 1 abgeschlossen ist, verliert der User („Game over“)
- Wenn die Zahl vorher erraten wird, gewinnt der User

```
Guess the number (between 1 and 100) as fast as you can!  
Your guess: 3  
Try a bigger number  
Your guess: 1  
Try a bigger number  
Game Over! Time's up. The correct number was: 29
```


BONUS: OBJEKTORIENTIERT

- Implementiere das gleiche Spiel **objektorientiert**:
 - Zwei Klassen `TimerThread(...)` und `UserInputThread()` erben von `threading.Thread`
 - Methode `run()` wird überschrieben
- *Vorteile*:
 - Strukturiert, wiederverwendbar, erweiterbar, ...

SYNCHRONISATION

Koordination von unabhängigen, nebenläufigen
Prozessen/Threads

SYNCHRONISATION

- Notwendigkeit zum exklusiven Zugriff auf Ressourcen/Daten
 - Daten bleiben konsistent
 - Gleichzeitiger Zugriff auf Gerät nicht möglich

Thread 1:
transfer(account1, account2, 200)

newbalance1 = 700

Thread 2:
transfer(account1, account2, 150)

newbalance1 = 650

```
def transfer(account1, account2, value):
```

```
    newbalance1 = account1.balance + value
```

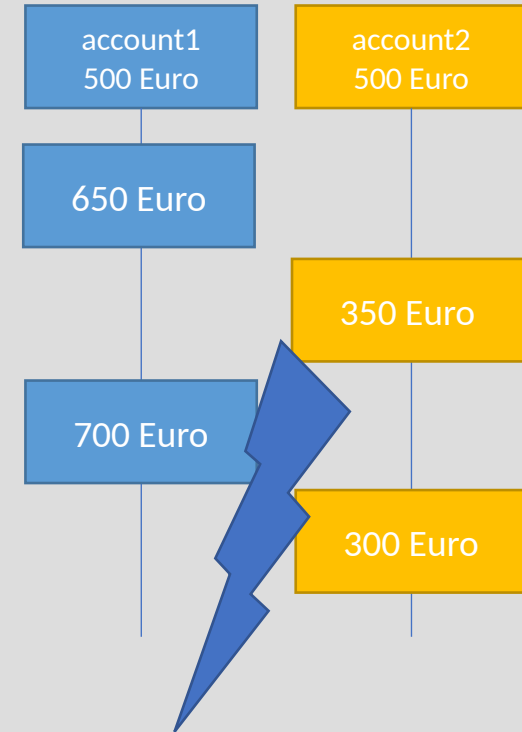
```
    newbalance2 = account2.balance - value
```

```
    account1.balance = newbalance1
```

```
    account2.balance = newbalance2
```

newbalance1 = 700
newbalance2 = 300

newbalance1 = 650
newbalance2 = 350



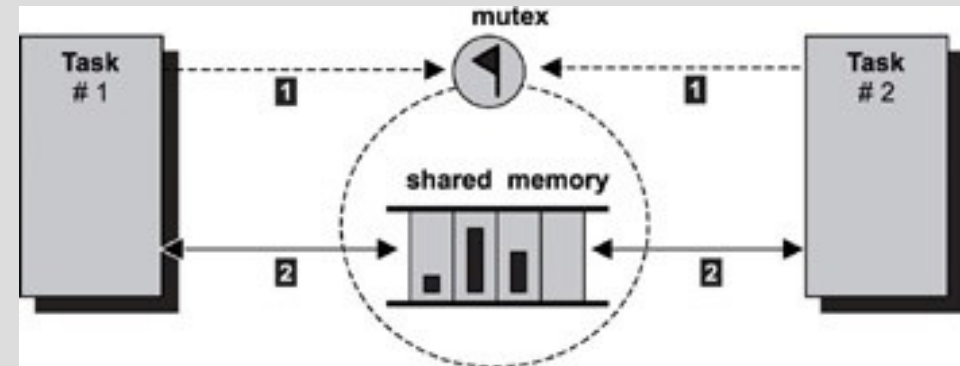
Eigentlich:
account1.balance: 850 Euro
account2.balance: 150 Euro

LOCK/MUTEX

- Garantiert, dass nur ein Prozess / Thread auf eine Ressource/Datenbereich zugreifen kann
 - Gegenseitiger Ausschluss
- Ziel: **Konsistenz**

```
1  import multiprocessing
2
3  mutex = multiprocessing.lock()
4
5  def transfer(acount1, acount2, value):
6      global mutex
7      mutex.acquire()
8      newbalance1 = acount1.balance + value
9      newbalance2 = acount2.balance - value
10     acount1.balance = newbalance1
11     acount2.balance = newbalance2
12     mutex.release()
```

11.01.2024



Quelle:

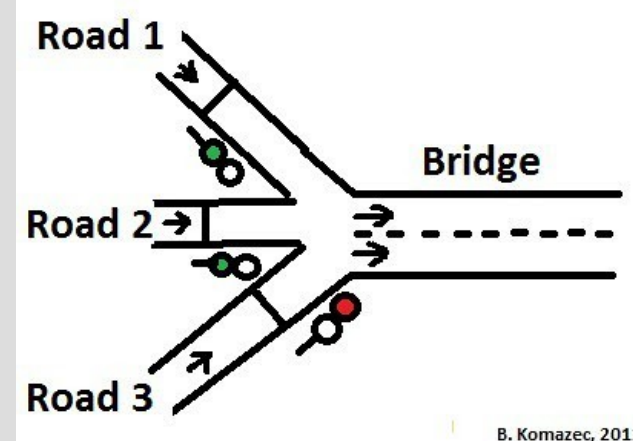
<http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0095.html>

Ähnlich wie Holz beim Staffellauf

Erweiterung auf Lese-/Schreib-Locks

SEMAPHORE

- Zugriffssperre für **mehrere Prozesse/Threads**
- Ähnlich wie Ampel-System
 - Wer darf die Ressource nutzen (Counter)
 - Wenn keine Ressource frei, dann warten
 - Meldung, wenn Möglichkeit auf Ressource zuzugreifen



```
1 import multiprocessing
2
3 sem = multiprocessing.Semaphore(2)
4
5 def run():
6     global sem
7     sem.acquire()
8     # use one of the two printer
9     sem.release()
```

=Zwei freie Plätze

SEMAPHORE EXMAMPLE

```
import threading
import time

# Funktion, die von Threads ausgeführt wird
def worker(sem, worker_id):
    print(f"Worker {worker_id} wartet auf das Semaphore.")

    # Warte auf das Semaphore
    sem.acquire()

    print(f"Worker {worker_id} hat das Semaphore erhalten und arbeitet.")
    time.sleep(2) # Simuliert die Arbeit des Threads

    # Freigabe des Semaphors nach Abschluss der Arbeit
    sem.release()
    print(f"Worker {worker_id} hat das Semaphore freigegeben.")
```

```
# Anzahl der verfügbaren Ressourcen (Anzahl der gleichzeitig erlaubten Threads)
num_resources = 2

# Erstellung des Semaphors
semaphore = threading.Semaphore(num_resources)

# Erstellung von Threads
threads = []
for i in range(5):
    t = threading.Thread(target=worker,
args=(semaphore, i))
    threads.append(t)

# Start der Threads
for t in threads:
    t.start()

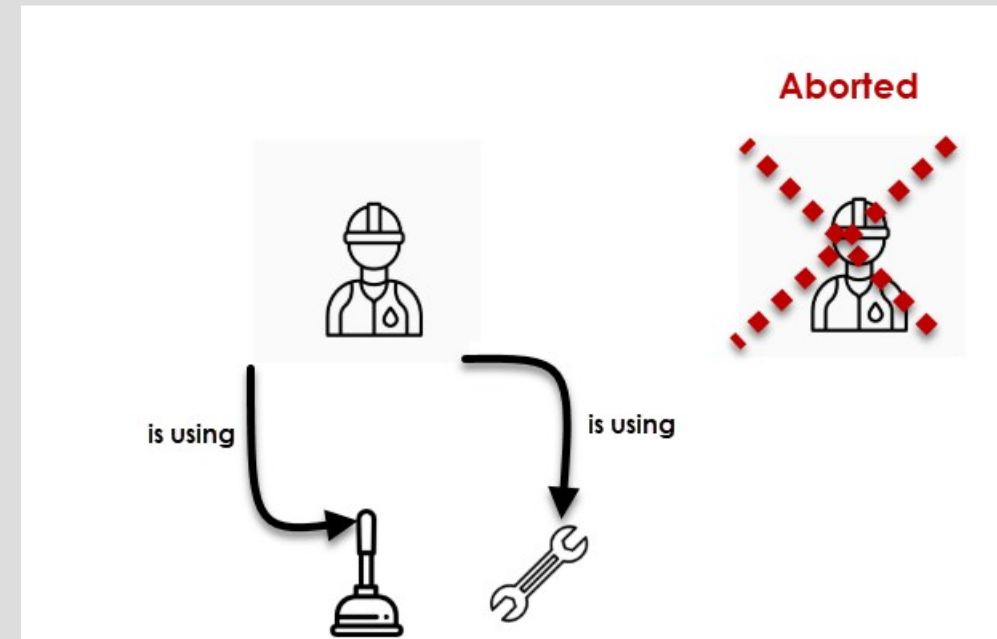
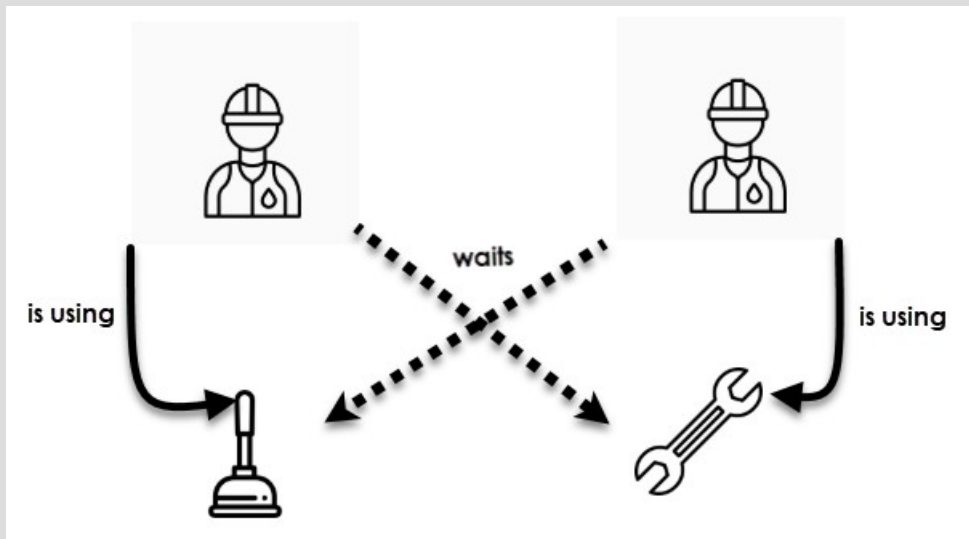
# Warten auf das Ende der Threads
for t in threads:
    t.join()

print("Alle Threads haben ihre Arbeit abgeschlossen.")
```

DEADLOCK

- **Definition:**

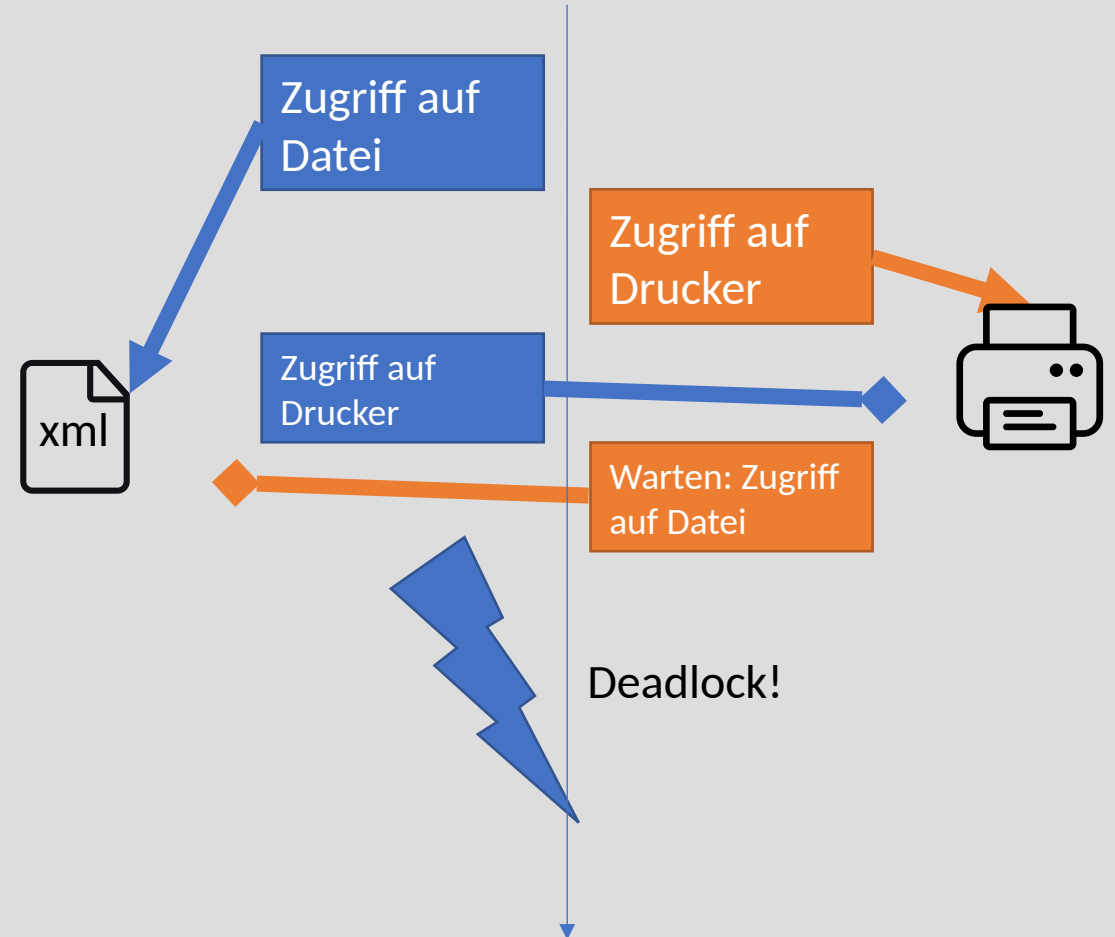
- Gegenseitige Blockade von mindestens zwei Prozessen
- Jeder Prozess wartet auf ein Ereignis/Zustand, welches von einem anderen beteiligten Prozess abhängt
- Alle warten aufeinander - endlos



DEADLOCK: BEISPIEL

Prozess A
sammelt Daten, aktualisiert damit die Datei
print_log.xml und druckt diese am Drucker aus.

Prozess B
druckt eine beliebige Datei am Drucker aus und
schreibt am Ende einen Eintrag in print_log.xml



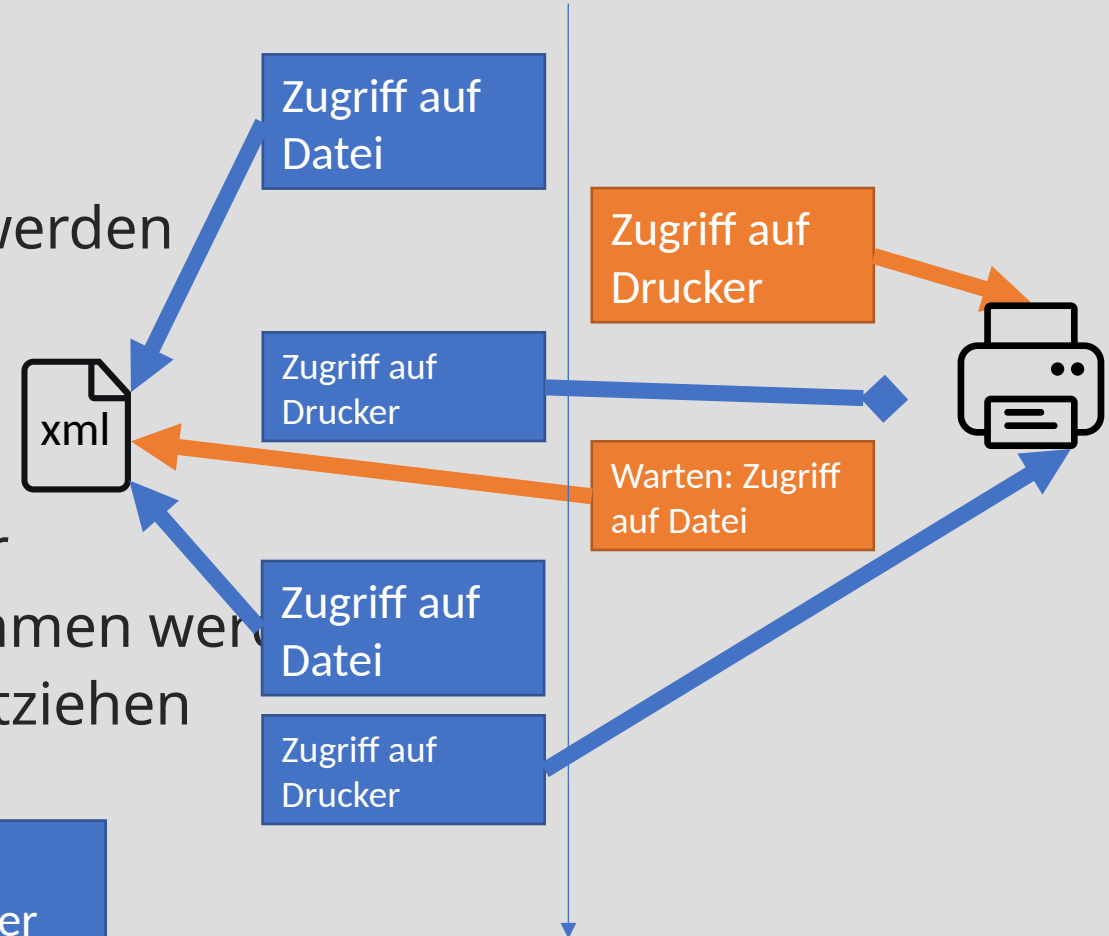
DEADLOCK

Voraussetzungen

- Exklusiver Zugriff
- Warten belegt Ressource
- Ressourcen können Prozess nicht entzogen werden
- Zyklisches Warten

• Lösungen

- z.B. Lesenden Zugriff immer erlauben
- kein Warten, wenn Ressource nicht verfügbar
- Ressourcen freigeben, wenn nicht alle bekommen werden
- Prozess Ressourcen nach bestimmter Zeit entziehen



Lösung für unser Beispiel:

beide Prozesse fordern sofort beide Ressourcen an. Wenn eine der beiden Ressourcen nicht verfügbar, wird die andere Ressource auch wieder frei gegeben.

INTER-PROCESS- COMMUNICATION

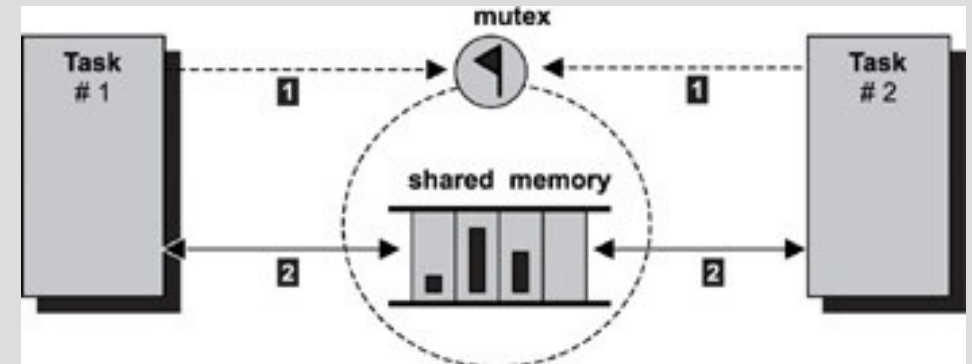
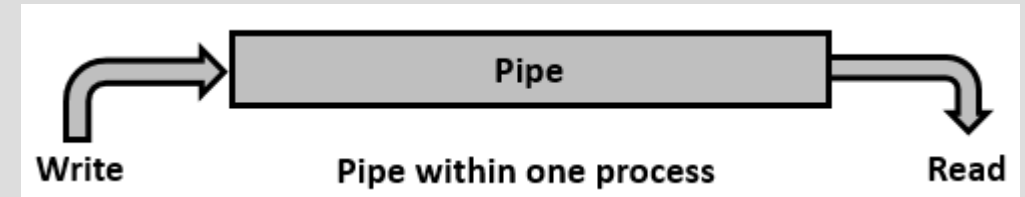
Kommunikation zwischen Prozessen

INTER-PROZESS-KOMMUNIKATION (IPC)

- Kommunikation zwischen Prozessen auf demselben Computer, deren Speicherbereiche aber strikt voneinander getrennt sind.
- **Problem:** Das Betriebssystem verhindert solche Zugriffe / Kommunikationen.
- Spezielle Lösungen nötig

ARTEN VON INTER-PROZESS-KOMMUNIKATION

- IPC kann erfolgen durch:
- Dateien
- Datenströme (Pipes):
 - Sequentiell
 - FIFO (Queue in multiprocessing)
 - Ein- und Ausgabe-Pipe
- Interrupts, Ereignisse, Nachrichten
- Shared Memory (mit Mutex-Mechanismus)
- Sockets
- Remote-Procedure-Calls (synchron, asynchron)

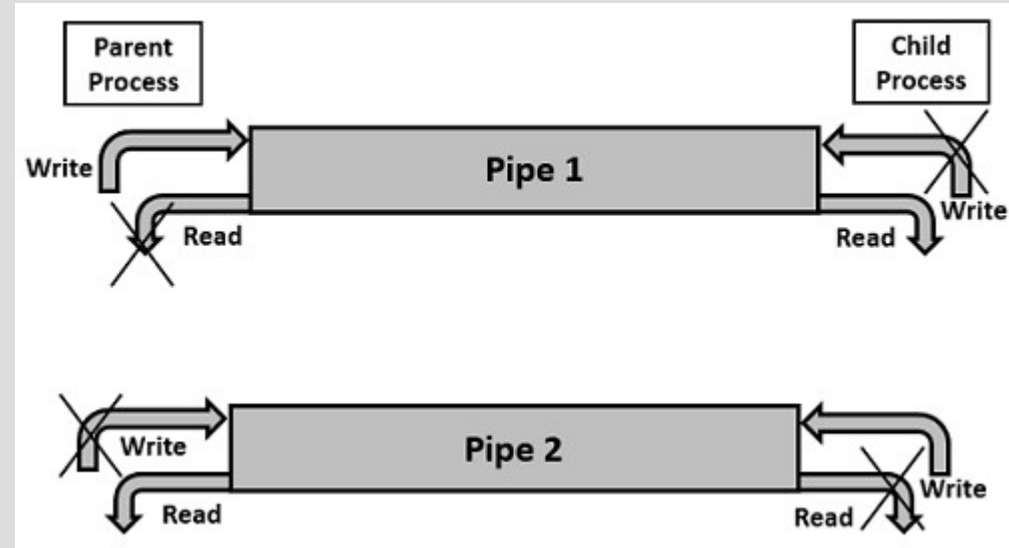


IPC IN PYTHON MITTELS PIPE

- **Ablauf:**

- Erzeugen **einer Pipe pro Prozess**
- Allen erzeugten Prozessen mitgeben
- Kommunikation über pipe
 - `pipe.recv()` – **Lesen** von Daten aus der Pipe
 - `pipe.send(...)` – **Senden** von Daten über die Pipe

- Abstraktion der Kommunikation in der Bibliothek **multiprocessing**



<https://spurin.com/2020/06/25/Python-Inter-Process-Communication/>

- In C: https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_pipes.htm

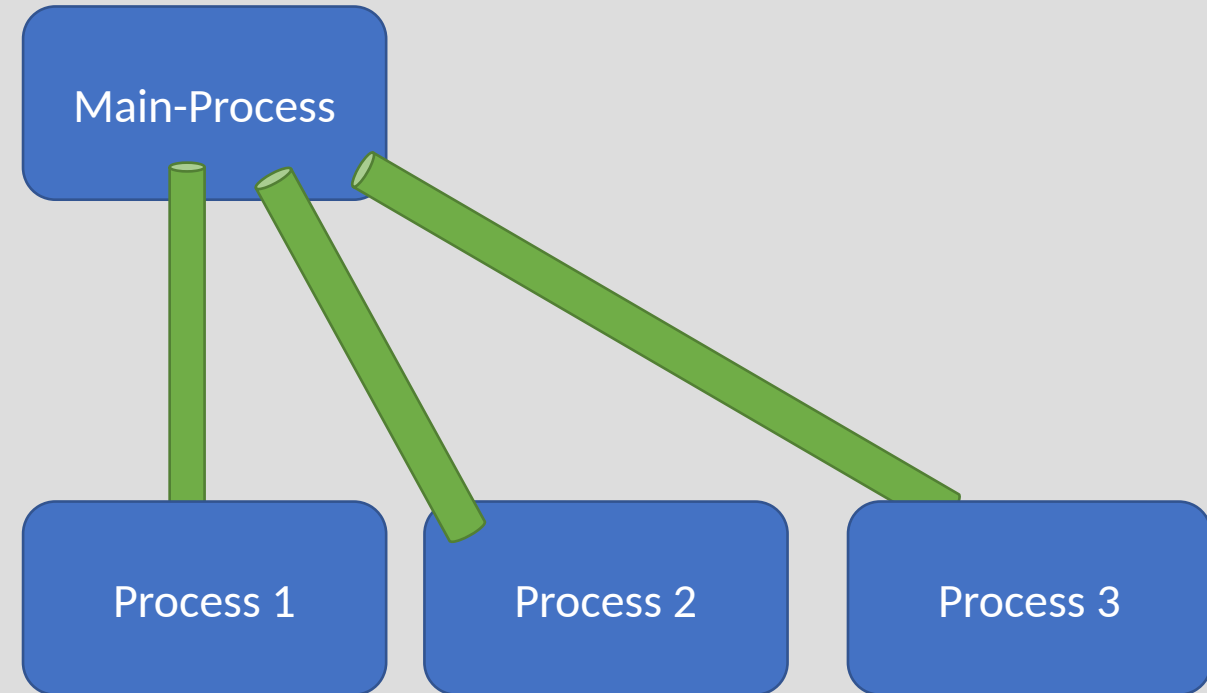
PIPES ZWISCHEN PROZESSEN

```
from multiprocessing import Process, Pipe

def calc_info(conn,i):
    conn.send([42,None,'hello '+str(i)])
    conn.close

if __name__=='__main__':
    for i in range(3):
        parent_conn,child_conn=Pipe()
        p=Process(target=calc_info,
        args=(child_conn, i))
        p.start()

        print(parent_conn.recv())
        p.join()
        parent_conn.close()
```



IPC IN PYTHON: VERWENDUNG VON POOL

```
import multiprocessing

def calculate_square(number):
    result = number * number
    print(f"The square of {number} is {result}")

if __name__ == "__main__":
    # Erstelle eine Liste von Zahlen
    numbers = [1, 2, 3, 4, 5]

    # Erstelle einen Pool von Prozessen
    with multiprocessing.Pool() as pool:
        # Führe die Funktion calculate_square für jede Zahl im Pool aus
        pool.map(calculate_square, numbers)
```

IPC IN PYTHON: MIT PIPES

```
import multiprocessing

def calculate_square(number, output_pipe):
    result = number * number
    output_pipe.send(result)

if __name__ == "__main__":
    # Erstelle eine Liste von Zahlen
    numbers = [1, 2, 3, 4, 5]

    # Erstelle eine Pipe für die
    # Datenkommunikation
    parent_pipe, child_pipe =
multiprocessing.Pipe()

    # Erstelle einen Pool von Prozessen
    with multiprocessing.Pool() as pool:
        # Führe die Funktion
        # calculate_square für jede Zahl im Pool aus
        # und sende das Ergebnis über die
        # Pipe
        results =
pool.starmap(calculate_square, [(num,
child_pipe) for num in numbers])

    # Lese die Ergebnisse aus der Pipe
    squared_numbers = [parent_pipe.recv()
for _ in results]

    # Ausgabe der quadrierten Zahlen
    print("Squared Numbers:",
squared_numbers)
```


AUFGABE

- Text-Datei mit **mehreren Prozessen** nach Text durchsuchen:
- **Idee:** Jeder Prozess bekommt einen Teil der Datei für die Suche
- Die gefundenen Positionen werden im Haupt-Prozess gesammelt und ausgegeben
- Einzelne Prozesse müssen dem Haupt-Prozess mitteilen, wenn sie fertig sind

Prozess 1

Prozess 2

Prozess 3

Prozess 4

Prozess 5

- Das ist ein Text in der Datei auf welchen mehrere Prozess zugreifen. Jeder Prozess bekommt einen anderen Teil der Datei zu lesen. So kann jeder Prozess in seinem Teil der Datei nach dem gesuchten Wort suchen. Es ist möglich, dass mehrere Prozesse auf die selbe Datei zugreifen, solange sie die Datei nur lesend öffnen. Hier am Ende steht nur mehr Blödsinn – z.B. schöne Ferien!

PIPES ZWISCHEN PROZESSEN

```
from multiprocessing import Process, Pipe

def calc_info(conn,i):
    conn.send([42,None,'hello '+str(i)])
    conn.close

if __name__=='__main__':
    for i in range(3):
        parent_conn,child_conn=Pipe()
        p=Process(target=calc_info,
        args=(child_conn, i))
        p.start()

        print(parent_conn.recv())
        p.join()
        parent_conn.close()
```

